TECHNICAL ARCHITECTURE & REFERENCE

Merchant Group & Merchant Operations

Source: Portal UAT Baseline | Generated: November 2026 | Scope: Merchant Portal Frontend & APIs

1. Executive Summary & Concepts

The **Merchant Group** architecture introduces a hierarchy allowing a single parent corporate entity (Group Administrator) to manage, monitor, and impersonate multiple affiliated merchant accounts registered under its umbrella.

Instead of logging in and out with different credentials for each individual store or subsidiary, a Merchant Group Admin logs into the portal using unified Group Admin credentials and seamlessly operates in either an **aggregated overview mode** or an **impersonated ("soft login") mode**.

Three Distinct Operating States

- **Standard Merchant:** A standalone business entity operating its own store, branches, cashiers, and financial fund transfers (Pay Bills, Transfer to Bank).
- **Merchant Group Admin (Aggregated View):** The admin entity logged in with Group credentials. Displays unified KPI totals, all linked accounts' transaction feeds, and the linked merchants registry. Sensitive merchant-specific actions are hidden.
- **Soft-Logged-in Merchant (Impersonated View):** The Group Admin temporarily switches context to act on behalf of a specific merchant. Can manage branches, cashiers, and reports, but **financial transfers (Pay Bills, Transfer to Bank) are strictly disabled for security**.

2. End-to-End User Journeys

```
[1. Merchant Registration Flow]
    Merchant Registers ──> Checks "Is merchant part of group?" ──> Enters Group ID
         ├── If Found: Displays group name ──> Sets groupId ──> Submits register API
         └── If Not Found: "Create New Group" Modal ──> Calls merchantGroup/add ──> Auto-selects new ID

[2. Group Admin Login Flow]
    Group Admin logs in (Email + Wallet PIN) ──> loginWithPin returns isMerchantGroup: true
         ├── Sets window.isMerchantGroup = true
         ├── getUserProfile stores data.merchantGroupInfo into window.customer
         └── getAccounts builds aggregated list: window.merchantGroupAccounts = [acc1, acc2, ...]

[3. Aggregated Overview Mode]
    Dashboard & Transactions pass accountId: window.merchantGroupAccounts (array)
         ├── Aggregated Balances & KPIs displayed
         ├── All linked merchants' live transactions visible
         └── Sidebar displays "Merchant Group" registry; hides individual merchant operations

[4. Merchant Impersonation (Soft Login)]
    Admin navigates to "Merchant Group" ──> Selects a merchant row ──> Clicks "Login"
         ├── Calls loginMerchantForGroup({ custId })
         ├── Sets window.isSoftLogin = true
         ├── Re-fetches getUserProfile (populates specific merchant profile)
         ├── Reveals cashier & branch management; hides Merchant Group tab
         ├── STRICT RESTRICTION: Hides Pay Bills & Transfer to Bank
         └── Displays "Back to Portal" button in header

[5. Return to Group Admin Mode]
    Admin clicks "Back to Portal" in header
         ├── Calls logoutMerchantForGroup({})
         ├── Deletes window.isSoftLogin
         ├── Re-fetches getUserProfile (restores group profile)
         └── Returns to Aggregated Dashboard view
```

3. Registration Flow: "Search by Unique Group ID"

Located in `pages/personal/register.html`, merchants can be linked to an existing group or initiate a new group during registration.

UI Field Mechanics

- **Group Checkbox (`#isMerchantGroup`):** Checkbox with label *"Is this merchant part of a group?"*. Toggling shows/hides `#merchantGroupSection`. When unchecked, it resets all group search inputs and errors.
- **Search Input (`#groupSearchId`):** Input with label *"Search by Unique Group ID"* and button *"Search"* (`#searchGroupBtn`). Supports typing + Enter key.
- **Hidden Form Field (`#groupId`):** Stores the validated `groupId`. Automatically collected by `validateCollectFormData('register-form')` into `window.pages_params['groupId']`, which is submitted in the payload to the `register` API.

Search Outcomes

Outcome	UI Behavior	Underlying Action
Group Found	Displays `#groupSearchResult` with `{groupId} : {groupName}` in green text.	Sets `selectedGroupId = records[0].groupId` and updates hidden input `#groupId`.
Group Not Found	Displays error *"Group not found."* and reveals the **"Create New Group"** button (`#createNewGroupBtn`).	Clears `selectedGroupId` and hidden input `#groupId`.
Create New Group Modal	Opens modal `#createGroupModal` collecting Group Name, Email, Wallet PIN, Confirm PIN, Address, Phone, Website, and Business Type.	Validates form & PIN match, then sends request to `merchantGroup/add`. On success, sets the created Group ID and auto-selects it.

4. Role-Based Menu & Navigation Matrix

Navigation in `pages/corporate/main_new.html` uses two CSS classes to toggle menu availability: `.hideForGroup` and `.isSoftLogin`.

Menu Item	CSS Classes	Standard Merchant	Group Admin (Aggregated)	Soft-Logged In (Acting)
Dashboard	*None*	Visible (Single)	Visible (Aggregated)	Visible (Merchant)
Business Profile	*None*	Visible (Merchant)	Visible (Group)	Visible (Merchant)
Live Transactions	*None*	Visible (Single)	Visible (Aggregated)	Visible (Merchant)
Merchant Group (`#mg_tab`)	`isSoftLogin`	Hidden	Visible	Hidden
Cashiers	`hideForGroup`	Visible	Hidden	Visible
Branches / Subsidiaries	`hideForGroup`	Visible	Hidden	Visible
Reports	`hideForGroup`	Visible	Hidden	Visible
Merchant Operations	`hideForGroup`	Visible	Hidden	Visible
Administration	`hideForGroup`	Visible	Hidden	Visible
Pay Bills	`hideForGroup` `isSoftLogin`	Visible	Hidden	Strictly Restricted
Transfer to Bank	`hideForGroup` `isSoftLogin`	Visible	Hidden	Strictly Restricted
Header: "Back to Portal"	`#logoutToMerchant`	Hidden	Hidden	Visible

> **Security Rule for Impersonated (Soft Login) Sessions**
>
> Features marked with both `hideForGroup` and `isSoftLogin` (specifically **Pay Bills** and **Transfer to Bank**) are permanently suppressed during a soft login. This guarantees that an administrative group viewer cannot initiate unauthorized monetary outflows on behalf of an impersonated merchant.

5. Complete API Catalog & Data Contracts

Endpoint	Trigger / Screen	Request Payload	Response Structure / Usage
`merchantGroup/search`	Clicking "Search" during registration	`{ groupId: string }`	`{ status, records: [{ groupId, groupName }] }` Finds group to link.
`merchantGroup/add`	Submitting "Create New Group" modal	`{ groupName, email, status, walletPin, confirmPin, languageId, mobilePhone, businessPhone, website, addrStreetNo, addrStreetName, unitName, city, state, country, zipCode, businessType, businessCategory }`	`{ status, records: { groupId } }` Creates group and returns assigned ID.
`register`	Submitting registration form	Standard merchant registration payload + `groupId: string`	`{ status: "success", message, statusCode: "0" }`
`loginWithPin`	Portal Login Screen	`{ username, password, pin, ... }`	`{ status: "success", isMerchantGroup: true }` Sets `window.isMerchantGroup = true`.
`getUserProfile`	Post-login & context switches	`{}`	If Group Admin: `data.merchantGroupInfo` If Soft-login / Merchant: `data` Sets `window.customer`.
`getAccounts`	Post-login account fetch	`{}`	`{ data: [{ ACCOUNTNUMBER, AVBALANCE }] }` In Group mode: sets `window.merchantGroupAccounts = [acc1, acc2, ...]`.
`getDashboardInfo`	Dashboard load & period filter	`{ period, accountId: window.merchantGroupAccounts }`	Aggregated metrics across all group merchants.
`transactionHistory`	Dashboard txns & Live Transactions	`{ accountId: window.merchantGroupAccounts, pageSize, pageNum, txnStatus }`	Aggregated transaction feed across all group merchants.
`getGroupMerchants`	Navigating to "Merchant Group" tab	`{}`	`{ records: [{ custId, custName, email, phCountryCode, mobilePhone, status, city, businessType, creationDate }] }`

Endpoint	Trigger / Screen	Request Payload	Response Structure / Usage
`loginMerchantForGroup`	Selecting merchant & clicking "Login"	`{ custId: string }`	`{ status: "success" }` Activates soft-login session context.
`logoutMerchantForGroup`	Clicking "Back to Portal" header button	`{}`	`{ status: "success" }` Terminates soft login; restores group session.
`closeStore`	Merchant Operations page	`{ subsidiaryID: string, operationDate: string }`	Closes store day and reconciles cashier till totals.

6. Merchant Operations Architecture

Merchant Operations (`pages/corporate/merchant_operations.html`) manages daily branch store statuses and cashier till balances:

- **Branch Selection:** Selects a branch (`#subsidiaryId`) and optional operation date.
- **Till Totals:** Displays current daily totals: Total Debit, Total Credit, Total Cash In, Total Cash Out across tills.
- **Store Closure:** Closes the store day via `closeStore` , resetting till and cash bin totals to zero for the next cycle.
- **Role Relationship:** Hidden for Group Admin in aggregated mode (since till closing is branch/store-specific), but accessible when soft-logged in as a specific merchant.

7. Key Implementation Files Reference

File Path (portal-uat)	Role & Responsibility
`pages/personal/register.html`	Group checkbox, Group search input, "Create New Group" modal, and `groupId` submission.
`pages/login.html`	Detects `data.isMerchantGroup === true` and flags `window.isMerchantGroup` .
`js/functions.js`	In `getUserProfile_response` , switches `window.customer` and builds `window.merchantGroupAccounts` .
`pages/corporate/main_new.html`	Sidebar menu visibility rules, `#logoutToMerchant` header button, and `backToPortal()` logic.
`pages/corporate/merchant_group.html`	Linked merchants registry table, row selection handler, and `loginToMerchant(custId)` execution.
`pages/corporate/dashboard_new.html`	Passes `window.merchantGroupAccounts` array to dashboard and transaction APIs.
`pages/corporate/merchant_operations.html`	Branch till balance monitoring and daily store closure (`closeStore`).